

深圳大学 计算机与软件学院

College of Computer Science and Software Engineering of Shenzhen University



系统编程

基于TaiShan服务器/openEuler OS 的实践

第二讲：进程间通信 – 信号

实验一学生实验报告评析

1. 有一个全局变量*i*初值为5，父进程对其进行加1操作，子进程看到的*i*的取值将会是多少？（10分）

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int i = 5;
    pid_t pid = fork();

    for(int k=0;k<3;k++){
        if (pid != 0) {
            printf("Parent has i = %d\n", ++i);
        }
        else {
            printf("Chile has i = %d\n", i);
        }
    }

    printf("Bye from process %d with i = %d\n", getpid(), i);
    return 0;
}
```

```
[wujunyi_2017192040@taishan02-vm-4 expl]$ ./Q3
Parent has i = 6
Parent has i = 7
Parent has i = 8
Bye from process 111967 with i = 8
Chile has i = 5
Chile has i = 5
Chile has i = 5
Bye from process 111968 with i = 5
[wujunyi_2017192040@taishan02-vm-4 expl]$
```

可见，父进程的 *i* 循环自增三次，最终输出 8，而子进程的 *i* 始终为 5，最终输出为 5。↵
进一步，查看父子进程中变量 *i* 的地址，如下所示：↵

```
[wujunyi_2017192040@taishan02-vm-4 expl]$ ./Q3
Parent has i = 6
Chile has i = 5
Parent has i = 7
Chile has i = 5
Parent has i = 8
Chile has i = 5
Bye from process 112088 with i = 8, Address:-624949020
Bye from process 112089 with i = 5, Address:-624949020
[wujunyi_2017192040@taishan02-vm-4 expl]$
```

由图 63，父子进程输出的 *i* 变量的地址却是相同的，然而二者 *i* 的数值却不相同。事实上，在 Linux 操作系统中，程序打印输出的变量地址通常为虚拟内存地址，通过内核映射到实际的物理内存。*i* 变量的值是独立的，即父子进程中 *i* 的物理地址是不一样的，所以物理内存中 *i* 变量应该为两个，两者都映射到了同一个虚拟内存地址。↵

信而有征

Talk is cheap, show me the code ...

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdint.h>

void mem_addr(unsigned long vaddr, unsigned long *paddr)
{
    int pageSize = getpagesize();

    unsigned long v_pageIndex = vaddr / pageSize;
    unsigned long v_offset = v_pageIndex * sizeof(uint64_t);
    unsigned long page_offset = vaddr % pageSize;
    uint64_t item = 0;

    int fd = open("/proc/self/pagemap", O_RDONLY);
    if (fd == -1)
    {
        printf("open /proc/self/pagemap error\n");
        return;
    }

    if (lseek(fd, v_offset, SEEK_SET) == -1)
    {
        printf("lseek error\n");
        return;
    }

    if (read(fd, &item, sizeof(uint64_t)) != sizeof(uint64_t))
    {
        printf("read item error\n");
        return;
    }

    if (((uint64_t)1 << 63) & item) == 0)
    {
        printf("page present is 0\n");
        return;
    }

    uint64_t phy_pageIndex = (((uint64_t)1 << 55) - 1) & item;

    *paddr = (phy_pageIndex * pageSize) + page_offset;
}

"mem_addr.c" [dos] 46L, 1084C
```

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

void mem_addr(unsigned long vaddr, unsigned long *paddr);

int main(int argc, char *argv[])
{
    int i = 5;

    unsigned long phyaddr;

    pid_t pid = fork();

    if (pid < 0) { perror("Fail to fork-\n"); exit(0); }

    if (pid > 0) {
        i++;
        unsigned long viraddrPtr = (unsigned long)&i;
        mem_addr(viraddrPtr, &phyaddr);
        printf("i = %d with virtual addr = %x and physical addr = %x\n", i, &i, phyaddr);
    } else {
        sleep(1);
        unsigned long viraddrPtr = (unsigned long)&i;
        mem_addr(viraddrPtr, &phyaddr);
        printf("i = %d with virtual addr = %x and physical addr = %x\n", i, &i, phyaddr);
    }
}

"variables.c" 32L, 739C
```

```
[szu@taishan02-vm-10 process]$ sudo ./variables
```

我们信任您已经从系统管理员那里了解了日常注意事项。
总结起来无外乎这三点：

- #1) 尊重别人的隐私。
- #2) 输入前要先考虑(后果和风险)。
- #3) 权力越大，责任越大。

```
[sudo] szu 的密码:
```

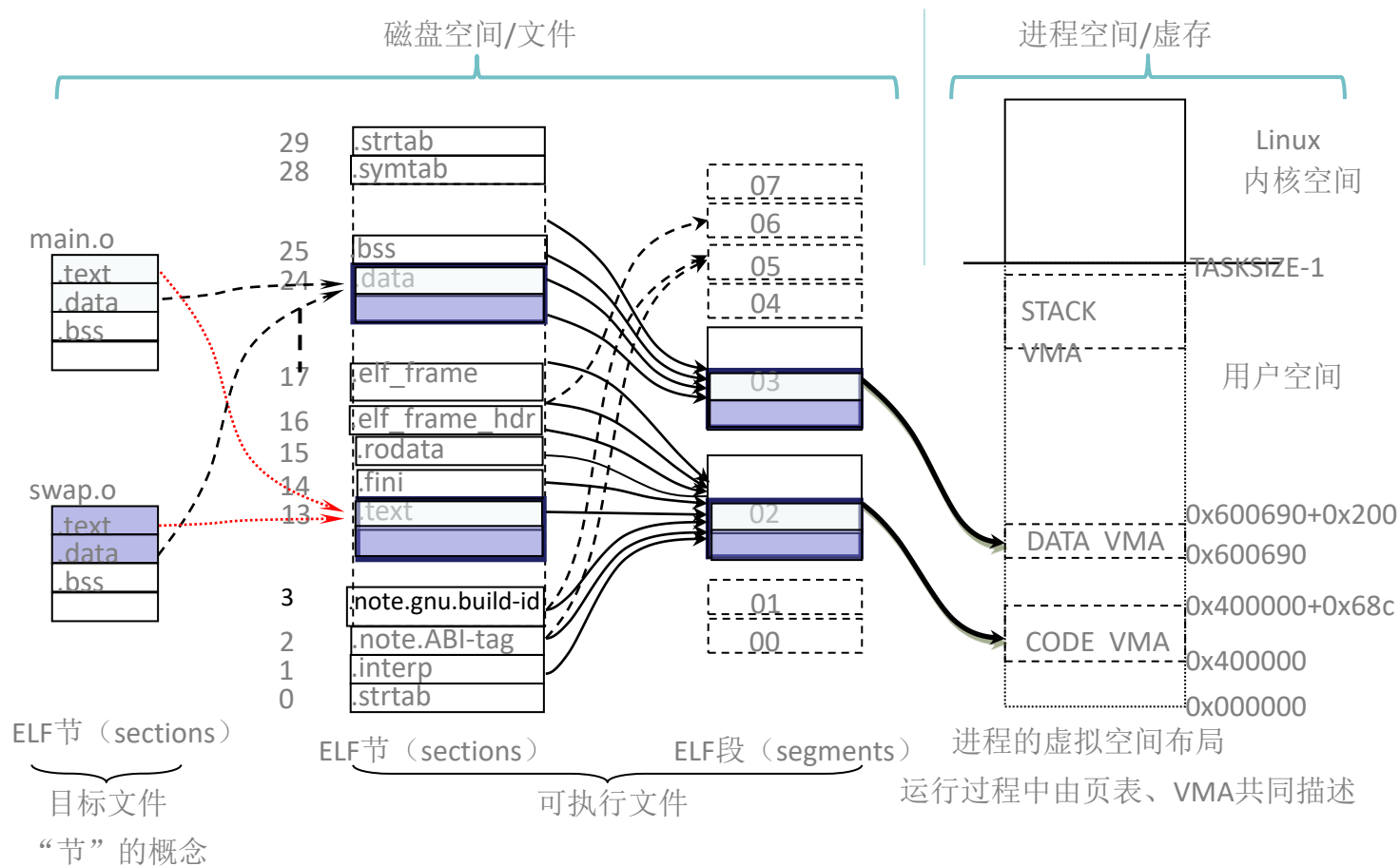
```
i = 6 with virtual addr = c4f75b44 and physical addr = 3e1c5b44
```

```
[szu@taishan02-vm-10 process]$ i = 5 with virtual addr = c4f75b44 and physical addr = 3e1d5b44
```

<https://blog.csdn.net/kongkongkkk/article/details/74366200>

《深入理解计算机系统》第四章

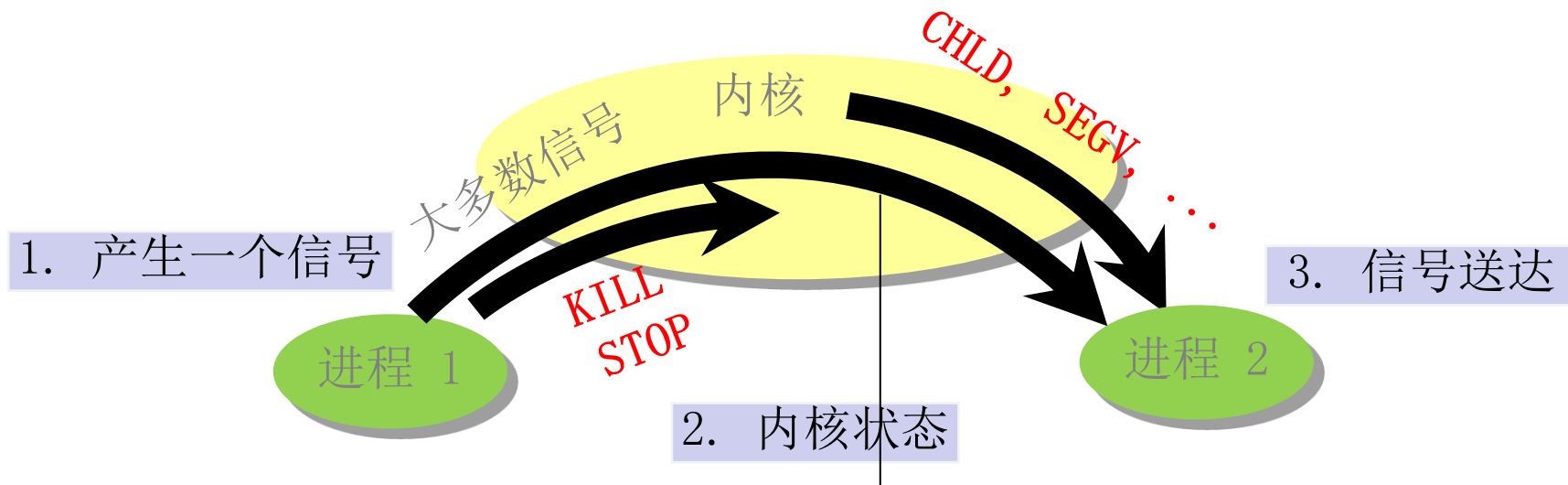
目标文件-
可执行文件-
进程影像
空间的关系



信号的基本概念

■ 信号(全称:软中断信号)是事件的异步通知

- 异步: 可以在任何时间发生
- 事件
 - ◆ 源于系统外部的事件: 用户产生的中断字符(Ctrl+C)
 - ◆ 源于程序或者内核的事件
 - 除以0
 - 空指针操作
- 信号的作用: 表示某种事件的发生(或者结束)



信号的基本概念

■ 有精确的生命周期

- (进程/内核)引发(raise)信号
- 内核存储该信号
- 传递信号到指定的进程

■ 信号的处理

- 忽略信号
 - ◆ 不做任何处理
 - ◆ 不能忽略的信号:SIGKILL && SIGSTOP
- 捕获并处理信号
 - ◆ 暂停进程的当前程序
 - ◆ 执行已注册的该信号的处理程序
 - ◆ 跳回进程暂停处继续执行
- 执行默认的动作

信号的基本概念

■ 信号标识符

- 信号标识符以SIG开头
- 定义在头文件
/usr/include/asm/signal.h
- 列出系统所支持信号列表
\$kill -l

```
#define SIGHUP 1
#define SIGINT 2
#define SIGQUIT 3
#define SIGILL 4
#define SIGTRAP 5
#define SIGABRT 6
#define SIGIOT 6
#define SIGBUS 7
#define SIGFPE 8
#define SIGKILL 9
#define SIGUSR1 10
#define SIGSEGV 11
#define SIGUSR2 12
#define SIGPIPE 13
#define SIGALRM 14
#define SIGTERM 15
#define SIGSTKFLT 16
#define SIGCHLD 17
#define SIGCONT 18
#define SIGSTOP 19
#define SIGTSTP 20
#define SIGTTIN 21
#define SIGTTOU 22
#define SIGURG 23
#define SIGXCPU 24
#define SIGXFSZ 25
#define SIGVTALRM 26
#define SIGPROF 27
#define SIGWINCH 28
#define SIGIO 29
#define SIGPOLL SIGIO
/*
#define SIGLOST 29
*/
#define SIGPWR 30
#define SIGSYS 31
#define SIGUNUSED 31

/* These should not be considered constants from userland. */
#define SIGRTMIN 32
#define SIGRTMAX _NSIG
```

man 7 signal

■ man

- 7: 杂项， 例如宏命令包、惯例等。

The entries in the "Action" column of the tables below specify the default disposition for each signal, as follows:

Term Default action is to terminate the process.

Ign Default action is to ignore the signal.

Core Default action is to terminate the process and dump core (see **core(5)**).

Stop Default action is to stop the process.

Cont Default action is to continue the process if it is currently stopped.

man 7 signal (继续)

First the signals described in the original POSIX.1-1990 standard.

Signal	Value	Action	Comment

SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from abort (3)
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from alarm (2)
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at tty
SIGTTIN	21,21,26	Stop	tty input for background process
SIGTTOU	22,22,27	Stop	tty output for background process

信号处理相关的系统调用

- 最简单最旧的信号管理接口:**signal**
- 设置发送信号的计时器**alarm**
- 高级信号处理:指定一个信号的处理函数:**sigaction**
- 修改当前的信号挡板**sigprocmask**
- 向一个进程发送信号**kill**

signal() 系统调用

- 定义函数并将其注册为指定信号的处理函数

```
#include <signal.h>
typedef void (*sighandler_t) (int);
sighandler_t signal(int signo, sighandler_t handler);
```

- /usr/include/bits/signum.h

```
/* Fake signal functions. */
#define SIG_ERR ((__sighandler_t) -1)          /* Error return. */
#define SIG_DFL ((__sighandler_t) 0)          /* Default action. */
#define SIG_IGN ((__sighandler_t) 1)          /* Ignore signal. */
```

signal() 系统调用

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signo, sighandler_t handler) {  
    if (handler == SIG_ERR) {  
        fprintf(stderr, " Cannot handle SIGINT!\n" );  
        exit(EXIT_FAILURE);  
    } else if (handler == SIG_DEL) {  
        .....  
    } else if (handler == SIG_IGN) {  
        .....  
    } else {  
        handler();  
    }  
    .....  
}
```

signal() 应用例子 (一)

文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
```

```
    while(1){
        printf("Hello World!\n");
        sleep(1);
    }
```

```
}
```

```
~
~
```

```
[yuhong@FedoraDVD13 signal]$ gcc -o noSignal noSignal.c
[yuhong@FedoraDVD13 signal]$ ./noSignal
Hello World!
Hello World!
Hello World!
^C
[yuhong@FedoraDVD13 signal]$
```

signal()应用例子（一）

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
```

```
void ouch(int sig)
{
    printf("OUCH!-I got signal%d\n",sig);

    //将 SIGINT的行为设定为默认值 ----进程将终止
    signal(SIGINT,SIG_DFL);
}
```

```
int main()
{
```

```
    //将 ouch注册为我们用于处理 SIGINT的信号处理函数
```

```
    signal(SIGINT,ouch);
```

```
    while(1){
```

```
        printf("Hello World!\n");
```

```
        sleep(1);
```

```
    }
```

```
}
```

```
~
```

```
[yuhong@FedoraDVD13 signal]$ gcc -o signal1 signal1.c
```

```
[yuhong@FedoraDVD13 signal]$ ./signal1
```

```
Hello World!
```

```
Hello World!
```

```
^COUCH!-I got signal2
```

```
Hello World!
```

```
Hello World!
```

```
Hello World!
```

```
^C
```

```
[yuhong@FedoraDVD13 signal]$ █
```

signal() 应用例子 (二)

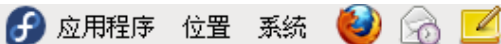
```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
```

```
void ouch(int sig)
{
    printf("OUCH!-I got signal%d\n",sig);
    //signal(SIGINT,SIG_DFL);
}
```

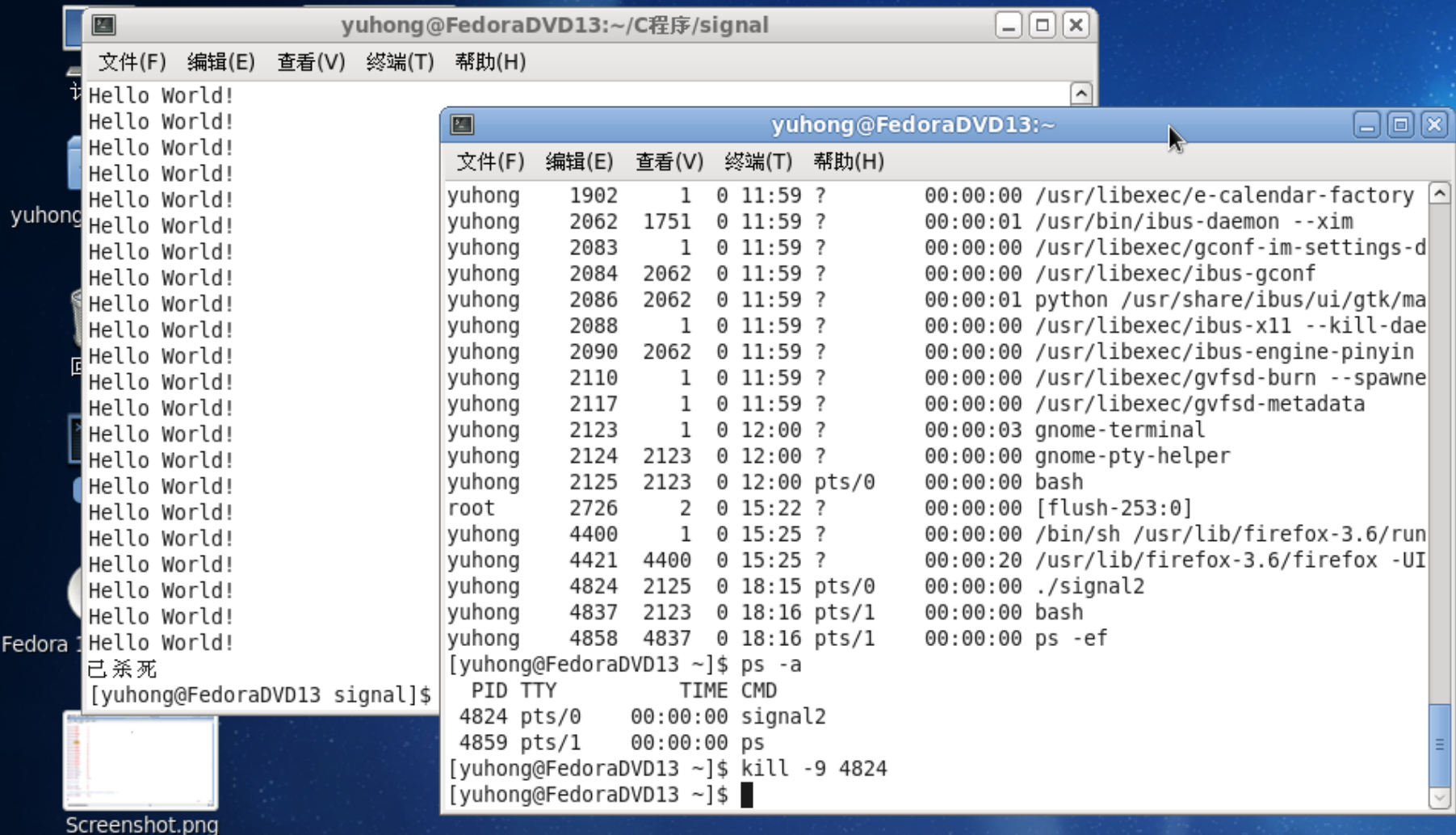
```
int main()
{
    signal(SIGINT,ouch);
    while(1){
        printf("Hello World!\n");
        sleep(1);
    }
}
```

```
[yuhong@FedoraDVD13 signal]$ gcc -o signal2 signal2.c
[yuhong@FedoraDVD13 signal]$ ./signal2
Hello World!
Hello World!
Hello World!
^COUCH!-I got signal2
Hello World!
^COUCH!-I got signal2
Hello World!
^COUCH!-I got signal2
Hello World!
^COUCH!-I got signal2
Hello World!
^COUCH!-I got signal2
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
```

signal()应用例子 (一)



yuhong feng



学生问题

王家满：在信号处理函数执行的时候，对于**errno**全局变量会不会产生可重入问题？

可重入函数 vs. 不可重入函数

■ 可重入函数

- 可被多个任务调用且不必担心数据出错的函数
- 可被中断：执行另一段代码返回不会出错
- 除了栈上的变量不依赖任何环境（包括static）
 - ◆ 全局变量采用互斥保护

■ 不可重入函数

- 使用了系统资源的函数
 - ◆ 全局变量
 - ◆ 中断向量表

.....

- 不能运行在多任务环境中

信号处理函数 & errno?

- 进入时保存
- 退出时恢复

以下函数很可能是不可重入函数

- 使用了静态的数据结构
 - getlogin()、gmtime()、getgrgid()
- 调用了动态内存分配/释放函数
 - ◆ malloc()
 - ◆ free()
- 调用了标准I/O函数
 - Why?

signal() 应用例子 (三)

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
```

```
static void sig_usr(int);
```

```
/*两个信号共用一个信号处理函数*/
```

```
int main()
```

```
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        perror("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        perror("can't catch SIGUSR2");

    for (;;)
        pause();
}
```

```
/*参数 signo 是信号编号*/
```

```
static void sig_usr(int signo)
```

```
{
    if (signo == SIGUSR1)
        printf("received SIGUSR1\n");
    else if (signo == SIGUSR2)
        printf("received SIGUSR2\n");
    else{
        printf("received signal: %d", signo);
        strerror(errno);
    }
    return;
}
```

```
[yuhong@FedoraDVD13 signal]$ ./userDefinedSignal &
[1] 27188
[yuhong@FedoraDVD13 signal]$ kill -USR1 27188
[yuhong@FedoraDVD13 signal]$ received SIGUSR1

[yuhong@FedoraDVD13 signal]$ kill -USR2 27188
[yuhong@FedoraDVD13 signal]$ received SIGUSR2

[yuhong@FedoraDVD13 signal]$ kill -USR3 27188
bash: kill: USR3: invalid signal specification
[yuhong@FedoraDVD13 signal]$
```

信号的产生

1. 进程调用 `kill(pid, signal)`

- 给进程 `pid` 发送一个信号 `signal`

2. 内核发出

- 子进程终止 (`SIGCHLD`)
- 违例内存访问 (`SIGSEGV`)
- ...

3. 命令行

`$kill -l` #列出系统理解的所有信号

`kill [-signal] pid`, 给进程`pid`发送信号

无条件终止进程`pid`的执行

`$kill -9 pid` #or

`$kill -KILL pid` #or

`$kill -SIGKILL pid`

终端交互的信号

```
yuhong@F
文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)
#include <stdio.h>

int main()
{
    while (1){}
```

■ Control-C: SIGINT

- 终止进程运行

■ Control-Z: SIGSTOP

- 暂停进程运行-不可忽略

■ Control-Y: SIGCONT

- 恢复暂停进程的运行

■ Control-\: SIGQUIT

- core dump

```
[yuhong@FedoraDVD13 ~]$ kill -9 3182
[yuhong@FedoraDVD13 signal]$ ./test
已杀死
```

```
[yuhong@FedoraDVD13 ~]$ kill -STOP 3235
[yuhong@FedoraDVD13 signal]$ ./test
```

```
[1]+  Stopped                  ./test
[yuhong@FedoraDVD13 signal]$
```

```
[yuhong@FedoraDVD13 ~]$ ps axj | grep PID; ps axj | grep test
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
2210	3245	3244	2210	pts/1	3244	S+	500	0:00	grep PID
2061	3235	3235	2061	pts/0	2061	T	500	5:23	./test
2210	3247	3246	2210	pts/1	3246	S+	500	0:00	grep test

```
[yuhong@FedoraDVD13 ~]$ kill -CONT 3235
```

```
[yuhong@FedoraDVD13 ~]$ ps axj | grep PID; ps axj | grep test
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
2210	3249	3248	2210	pts/1	3248	S+	500	0:00	grep PID
2061	3235	3235	2061	pts/0	2061	R	500	5:27	./test
2210	3251	3250	2210	pts/1	3250	S+	500	0:00	grep test

```
[yuhong@FedoraDVD13 ~]$ ps axj | grep PID; ps axj | grep test
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
2210	3255	3254	2210	pts/1	3254	S+	500	0:00	grep PID
2061	3252	3252	2061	pts/0	3252	R+	500	0:27	./test
2210	3257	3256	2210	pts/1	3256	S+	500	0:00	grep test

```
[yuhong@FedoraDVD13 ~]$ kill -QUIT 3252
```

```
[yuhong@FedoraDVD13 signal]$ ./test
[1]+  退出                      (core dumped) ./test
退出 (core dumped)
```

进程也可给自己发信号

■ 类似引发异常

- `raise(signal)` or

- `kill(getpid(), signal)`

■ 等待指定时间后发送SIGALRM信号

- `unsigned alarm(unsigned seconds);`

 - `alarm(20)`

 - 20秒后发送SIGALRM 给调用进程

- 前面调用的`alarm()`如果还没有到时间将被取消

 - `alarm(0)`

 - 取消之前的`alarm()`

信号的三种状态

- 产生(generated)
- 等待(pending, 未决):在产生和到达之间的状态
 - 目标进程可阻止信号(block)
 - ◆ 解除对该信号的阻止
 - 目标进程可忽略该信号
- 送达(delivered)



3. 172.31.234.200 (szu)



```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <signal.h>
4 static void sigusr1_handler(int signo)
5 {
6     static int revcount = 1;
7     printf("Receive siguser1 %d times.\n", revcount++);
8 }
9 int main(int argc, char *argv[])
10 {
11     if (signal(SIGUSR1, sigusr1_handler) == SIG_ERR){
12         perror("Fail to reg sigusr1");
13         exit(EXIT_FAILURE);
14     }
15     for (int i = 0; i < 5; i++){
16         raise(SIGUSR1);
17         printf("Send a SIGUSR1.\n");
18     }
19     return 0;
20 }
```

```
[szu@taishan02-vm-10 sys_signal]$ ./intra
Receive siguser1 1 times.
Send a SIGUSR1.
Receive siguser1 2 times.
Send a SIGUSR1.
Receive siguser1 3 times.
Send a SIGUSR1.
Receive siguser1 4 times.
Send a SIGUSR1.
Receive siguser1 5 times.
Send a SIGUSR1.
[szu@taishan02-vm-10 sys_signal]$
```



```
1 #include <stdlib.h>
2 #include <signal.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 pid_t Fork(void)
6 {
7     pid_t ret = fork();
8     if (ret < 0) {
9         perror("Fail to fork in Fork()");
10        return -1;
11    }
12    return ret;
13 }
14 static void sigusr1_handler(int signo)
15 {
16     static int revcount = 1;
17     printf("Receive siguser1 %d times.\n", revcount++);
18 }
19 int main(int argc, char *argv[])
20 {
21     int i;
22     if (signal(SIGUSR1, sigusr1_handler) == SIG_ERR){
23         perror("Fail to reg sigusr1");
24         exit(EXIT_FAILURE);
25     }
26     if (Fork()) { while(1); }
27     else {
28         for (i = 0; i < 5; i++){
29             kill(getppid(), SIGUSR1);
30             printf("send SIGUSR1\n");
31         }
32     }
33 }
```

```
[szu@taishan02-vm-10 exam]$ ./inter
Receive siguser1 1 times.
send SIGUSR1
send SIGUSR1
Receive siguser1 2 times.
send SIGUSR1
Receive siguser1 3 times.
send SIGUSR1
Receive siguser1 4 times.
send SIGUSR1
Receive siguser1 5 times.
^C
[szu@taishan02-vm-10 exam]$ ./inter
send SIGUSR1
Receive siguser1 1 times.
send SIGUSR1
send SIGUSR1
send SIGUSR1
Receive siguser1 2 times.
send SIGUSR1
Receive siguser1 3 times.
^C
[szu@taishan02-vm-10 exam]$
```

信号掩码

■ 暂时阻止指定类型信号的送达

- 以位矢量方式实现
- 与内核挂起和阻止信号送达的表示方式相同

问题:

1. 位矢量的0和1的含义?
2. 各信号与sig bits之间是如何映射的?

SigInt	SigQuit	SigKill	...	SigCont	SigAbrt
1	0	1	...	1	0

■ 阻止所有的信号:

```
sigset_t sigs; //typedef unsigned long sigset_t
sigfillset(&sigs); //初始化sigs信号集, 使其包含所有的信号
sigprocmask(SIG_SETMASK, &sigs, NULL);
```

■ 除了sigfillset, 你也可以尝试:

- sigemptyset(&sigs) //初始化sigs信号集, 清除其所有的信号
- sigaddset(&sigs, signo) //将信号signo添加到信号集sigs中
- sigdelset(&sigs, signo) //从信号集sigs中删除信号signo
- sigismember(&sigs, signo) //测试信号signo是否已加入信号集sigs

```
/usr/include/bits/types/sigset_t.h
```

```
/usr/include/bits/types/  sigset  t.h
```

sigset.c

```

#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
static void sigusr1_handler(int signo);

int main()
{
    sigset_t oldmask, newmask, pendmask;

    if (signal(SIGUSR1, sigusr1_handler) == SIG_ERR){printf("signal error\n");exit(1);}

    sigemptyset(&newmask); sigaddset(&newmask, SIGUSR1);
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0) { printf("sigprocmask block error\n"); exit(1);}
    printf("SIGUSR1 is blocked\n");

    int i;
    for (i = 0; i < 5; i++){raise(SIGUSR1); printf("send SIGUSR1\n");}

    printf("sleep started\n");sleep(5);printf("sleep finished\n");

    if (sigpending(&pendmask)<0){printf("sigpending error\n"); exit(1);}
    if (sigismember(&pendmask, SIGUSR1)) { printf("SIGUSR1 was pending\n");}

    if (sigprocmask(SIG_SETMASK, &oldmask, NULL)<0){printf("sigprocmask setmask error\n"); exit(1);}
    printf("restore old signal mask\n");
    exit(0);
}

static void sigusr1_handler(int signo)
{
    static int n = 1;
    printf("receive user1 signal %d times\n", n++);
    if (signal(SIGUSR1, sigusr1_handler) == SIG_ERR) { printf("can't reset SIGUSR1");exit(1);}
}

```

"multiBlockedSignal.c" 34L, 1059C

SIGUSR1 is blocked
 send SIGUSR1
 send SIGUSR1
 send SIGUSR1
 send SIGUSR1
 send SIGUSR1
 sleep started
 sleep finished
 SIGUSR1 was pending
 receive user1 signal 1 times
 restore old signal mask

```

int main()
{
    sigset_t oldmask,newmask,pendmask;

    if (signal(SIGUSR1,sigusr1_handler) == SIG_ERR){printf("signal error\n");exit(1);}
    if (signal(SIGUSR2,sigusr1_handler) == SIG_ERR){printf("signal error\n");exit(1);}

    sigemptyset(&newmask); sigaddset(&newmask,SIGUSR1);
    sigaddset(&newmask,SIGUSR2);

    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0) { printf("sigprocmask block error\n"); exit(1);}
    printf("SIGUSR1 and SIGUSR2 are blocked\n");

    int i;
    for (i = 0; i < 5; i++){raise(SIGUSR2); printf("send SIGUSR2\n");}
    for (i = 0; i < 5; i++){raise(SIGUSR1); printf("send SIGUSR1\n");}
    for (i = 0; i < 5; i++){raise(SIGUSR2); printf("send SIGUSR2\n");}

    printf("sleep started\n");sleep(5);printf("sleep finished\n");

    if (sigpending(&pendmask)<0){printf("sigpending error\n"); exit(1);}
    if (sigismember(&pendmask,SIGUSR1)) { printf("SIGUSR1 was pending\n");}
    if (sigismember(&pendmask,SIGUSR2)) { printf("SIGUSR2 was pending\n");}

    if (sigprocmask(SIG_SETMASK,&oldmask,NULL)<0){printf("sigprocmask setmask error\n"); exit(1);}
    printf("restore old signal mask\n");
    exit(0);
}

```

```

static void sigusr1_handler(int signo)
{
    static int n = 1;
    if (signo == SIGUSR1){
        printf("receive user1 signal %d times\n",n++);
    } else if (signo == SIGUSR2){
        printf("receive user2 signal %d times\n",n++);
    } else {
        printf("received signal: %d\n",signo);
        strerror(errno);
    }
}

```

```

SIGUSR1 and SIGUSR2 are blocked
send SIGUSR2
send SIGUSR2
send SIGUSR2
send SIGUSR2
send SIGUSR2
send SIGUSR1
send SIGUSR1
send SIGUSR1
send SIGUSR1
send SIGUSR1
send SIGUSR2
send SIGUSR2
send SIGUSR2
send SIGUSR2
send SIGUSR2
sleep started
sleep finished
SIGUSR1 was pending
SIGUSR2 was pending
receive user2 signal 1 times
receive user1 signal 2 times
restore old signal mask

```

信号的产生、等待以及到达

■ 信号的可靠性：信号是否会丢失, 是否支持排队

- SIGHUP (1号) ~SIGSYS(31号): 不可靠
 - ◆ 信号在解除阻止之前引发了多次, 只算一次。
- SIGRTMIN(33号)~SIGRTMAX(64号):可靠
 - ◆ POSIX.4上增加实时扩展功能
 - ◆ 支持多个同种pending信号的队列化处理

从内核信号模型解释

```
struct task_struct {  
    ...  
    //信号描述符  
    struct signal_struct *signal;  
    //信号处理描述符  
    struct sighand_struct *sighand;  
    //阻塞信号的掩码  
    sigset_t blocked;  
    sigset_t real_blocked;  
    //调用set_restore_sigmask()时  
    //恢复信号掩码  
    sigset_t saved_sigmask;  
    //未决信号列表  
    struct sigpending pending;  
    //指定特定信号处理栈空间,  
    //默认使用用户态  
    unsigned long sas_ss_sp;  
    size_t sas_ss_size;  
    unsigned int sas_ss_flags;  
    ...  
}
```

```
struct sighand_struct {  
    atomic_t count;  
    struct k_sigaction action[_NSIG];  
    spinlock_t siglock;  
    wait_queue_head_t signalfd_wqh;  
};
```

```
struct sigqueue {  
    struct list_head list;  
    int flags;  
    siginfo_t info;  
    struct user_struct *user;  
};
```

sigqueue

```
struct sigqueue {  
    struct list_head list;  
    int flags;  
    siginfo_t info;  
    struct user_struct *user;  
};
```

```
struct sigpending {  
    //未决可靠信号队列  
    struct list_head list;  
    // 未决信号集  
    sigset_t signal;  
};
```


alarm() 应用例子（一）

```
int main(void) {  
    alarm(10);  
    while(1);  
}
```

- 进程给自己发信号
- “无限” 循环，仅10秒
- 被闹钟中断
 - 程序没有注册信号处理函数
 - 缺省行为：终止进程

alarm() 应用例子（二）

```
#include <stdlib.h>
```

```
#include <signal.h>
```

```
int main(int argc, char** argv) {  
    while (1) {  
        if (fork())  
            sleep(30);  
        else  
            kill(getppid(), SIGKILL);  
    }  
}
```



```

/* unkillable.c */
1 #include <stdlib.h>
2 #include <signal.h>
3 #include <unistd.h>
4 int main(int argc, char *argv[])
5 {
6     while(1){
7         if (fork()) {
8             sleep(30);
9         } else {
10             kill(getppid(), SIGKILL);
11         }
12     }
13 }

```

```

1 $date; ps axj | grep unkillable
2 2022 年 01 月 24 日 星期一 15:46:17 CST
3 1 1444639 1669120 1666465 pts/1 1444637 S 0 0:00 ./unkillable
4 1444639 1444640 1669120 1666465 pts/1 1444637 R 0 0:00 ./unkillable
5 $date; ps axj | grep unkillable
6 2022 年 01 月 24 日 星期一 15:46:17 CST
7 1 1446608 1669120 1666465 pts/1 1446606 S 0 0:00 ./unkillable
8 1446608 1446609 1669120 1666465 pts/1 1446606 R 0 0:00 ./unkillable
9 $date; ps axj | grep unkillable
10 2022 年 01 月 24 日 星期一 15:46:18 CST
11 1 1448895 1669120 1666465 pts/1 1448893 S 0 0:00 ./unkillable
12 1448895 1448896 1669120 1666465 pts/1 1448893 R 0 0:00 ./unkillable
13 $date; ps axj | grep unkillable
14 2022 年 01 月 24 日 星期一 15:46:19 CST
15 1 1454534 1669120 1666465 pts/1 1454535 S 0 0:00 ./unkillable
16 1454534 1454537 1669120 1666465 pts/1 1454535 R 0 0:00 ./unkillable
17 $date; ps axj | grep unkillable
18 2022 年 01 月 24 日 星期一 15:46:20 CST
19 1 1459293 1669120 1666465 pts/1 1459291 S 0 0:00 ./unkillable
20 1459293 1459294 1669120 1666465 pts/1 1459291 R 0 0:00 ./unkillable

```

2021年

2011 ~ 2019: 学校的台式机 + Vmware + Fedora, 一直是这个结果

```

/* unkillable.c */
1 #include <stdlib.h>
2 #include <signal.h>
3 #include <unistd.h>
4 int main(int argc, char *argv[])
5 {
6     while(1) {
7         if (fork()) {
8             sleep(30);
9         } else {
10             kill(getppid(), SIGKILL);
11         }
12     }
13 }

```

```

1 $. /unkillable
2 已杀死
3 $date; ps axj | grep unkillable
4 2021 年 12 月 28 日 星期二 10:22:31 CST
5 2660041 288096 288095 2660041 pts/1 288095 R+ 1000 0:00 grep --color=auto unkillable
6 $date; ps axj | grep unkillable
7 2021 年 12 月 28 日 星期二 10:22:57 CST
8 2660041 450399 450396 2660041 pts/1 450396 S+ 1000 0:00 grep --color=auto unkillable

```

```

9 $date; ps axj | grep unkillable
10 2021 年 12 月 28 日 星期二 10:23:04 CST
11 1 495506 267300 2660041 pts/1 495504 S 1000 0:00 ./unkillable
12 2660041 495507 495504 2660041 pts/1 495504 S+ 1000 0:00 grep --color=auto unkillable
13 495506 495508 267300 2660041 pts/1 495504 R 1000 0:00 ./unkillable
14 $date; ps axj | grep unkillable
15 2021 年 12 月 28 日 星期二 10:23:11 CST
16 2660041 538020 538018 2660041 pts/1 538018 R+ 1000 0:00 grep --color=auto unkillable
17 $date; ps axj | grep unkillable
18 2021 年 12 月 28 日 星期二 10:23:40 CST
19 2660041 711462 711460 2660041 pts/1 711460 R+ 1000 0:00 grep --color=auto unkillable
20 $date; ps axj | grep unkillable
21 2021 年 12 月 28 日 星期二 10:23:45 CST
22 2660041 743443 743440 2660041 pts/1 743440 R+ 1000 0:00 grep --color=auto unkillable
23 1 743445 267300 2660041 pts/1 743440 R 1000 0:00 ./unkillable
24 743445 743446 267300 2660041 pts/1 743440 Z 1000 0:00 [unkillable] <defunct>
25 $date; ps axj | grep unkillable
26 2021 年 12 月 28 日 星期二 10:24:47 CST
27 2660041 1112815 1112813 2660041 pts/1 1112813 R+ 1000 0:00 grep --color=auto unkillable
28 $date; ps axj | grep unkillable
29 2021 年 12 月 29 日 星期三 10:03:02 CST
30 1 1687880 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
31 1312194 1687881 1687879 1312194 pts/0 1687879 S+ 1000 0:00 grep --color=auto unkillable
32 1 1687882 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
33 1 1687883 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
34 1 1687884 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
35 1 1687885 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
36 1 1687886 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
37 1 1687887 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
38 1 1687888 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
39 1 1687889 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
40 1 1687890 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
41 1 1687891 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>
42 1 1687892 1313941 1312194 pts/0 1687879 Z 1000 0:00 [unkillable] <defunct>

```

在我们配置的Taishan服务器虚拟机+ openEuler上运行是这样的。。。

3点奇怪的状态:

- (i) ps 不是总能看到该程序运行期生成的进程;
- (ii) 父进程生成子进程后, 调用sleep(30)会让出处理机, 子进程调用kill()函数会给父进程发送SIGKILL信号让其终止。但有些时刻存在子进程是僵尸进程, 父进程是运行状态的情况。这些父进程最终还是会收到其子进程发给它的SIGKILL信号并终止了运行;
- (iii) 存在瞬间大量僵尸进程的情况;

我有一个很好的助教-黎浩然，用3个系统来查看运行结果：

openEuler、ubuntu、苹果

```
root@Ernestdewindows: ~  
3630 pts/1 00:00:00 ps  
3660 pts/1 00:00:00 unkillable <defunct>  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
15712 pts/1 00:00:00 ps  
15745 pts/1 00:00:00 unkillable <defunct>  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
21336 pts/1 00:00:00 bash  
28611 pts/1 00:00:00 ps  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
10624 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
21336 pts/1 00:00:00 bash  
22197 pts/1 00:00:00 ps  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
3287 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
15177 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
21336 pts/1 00:00:00 bash  
26474 pts/1 00:00:00 ps  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
6060 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
19562 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
20486 pts/1 00:00:00 ps  
20517 pts/1 00:00:00 unkillable <defunct>  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~# ps  
PID TTY TIME CMD  
14318 pts/1 00:00:00 ps  
21336 pts/1 00:00:00 bash  
root@Ernestdewindows:~#
```

在ubuntu上的运行结果

```
1504 ttys000 0:00.02 -bash  
12958 ttys000 0:00.00 (unkillable)  
12959 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.02 -bash  
14332 ttys000 0:00.00 (unkillable)  
14333 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.02 -bash  
15540 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.02 -bash  
16985 ttys000 0:00.00 (unkillable)  
16986 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
18233 ttys000 0:00.00 (unkillable)  
18234 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
20225 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
21572 ttys000 0:00.00 (unkillable)  
21573 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
23643 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
25343 ttys000 0:00.00 (unkillable)  
25344 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
43534 ttys000 0:00.00 (unkillable)  
43535 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
45199 ttys000 0:00.00 (unkillable)  
45200 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.03 -bash  
47101 ttys000 0:00.00 (unkillable)  
MacBook-Pro:~ ernest$ ps  
PID TTY TIME CMD  
1504 ttys000 0:00.04 -bash  
56995 ttys000 0:00.00 (unkillable)  
56996 ttys000 0:00.00 (unkillable)
```

在macOS上的运行结果

10月8日深大冯老师技术交流 (5)

2022年1月10日 16:13



高桥 (华为openEuler)

@长恣

技术分析进展，可以在群里面给大家同步一下



长恣

好，我马上同步一下

10天后……

校企协同、软硬件协同分析问题

2022年1月10日 17:34



长恣

1. 针对第一个现象：ps命令不能总能观察到生成的进程，进程出现是周期性；这个问题应该和进程调度有关系。针对这个程序，所有进程最终都是由1号进程回收的，unkillable进程不断产生子进程同时杀死父进程。在一定的时间里，1号进程分配的时间片较多，其回收的速度大于进程消亡速度，特定的某个进程处于运行状态 (R)和僵尸状态 (Z)的时间极短，ps命令不一定抓的到（原理是根据/proc/PID/stat status cmdline三个文件来解析）。如果在某个时间段内，1号进程分配的时间片较少，僵尸状态的进程来不及回收，此时ps命令就会抓到unkillable进程。

（注：出现这种情况，是因为cpu核心数较少，进程会争抢cpu资源，在物理机上测试时，因为核数较多，1号进程几乎可以独享一颗cpu核心，出现僵尸进程会立刻被回收，此时ps命令完全无法抓到unkillable进程。关闭物理机核心，保留2个核心，可以浮现此问题）

2. 针对现象3：瞬间出现大量的僵尸进程

这个问题需要跟您确认，是否存在多次启动unkillable程序的情况？

在测试时，我们发现killall命令并不一定能杀掉unkillable进程，如果这个时候再启动一次unkillable程序，僵尸进程产生的速度会变快，1号进程回收速度如果更不上，这时ps会抓到大量的僵尸进程。

在多核物理机上实测时，因为cpu资源充足，unkillable进程完全不能被killall命令杀死。核数少的时候，多次执行killall命令，才能完全杀死unkillable进程。

3. 针对现象2：存在父进程时R收到子进程SIGKILL信号，仍处于R状态，子进程已经被孙子进程杀死，处于Z状态。

从原理上看，进程收到SIGKILL信号，再通过系统调用来终结进程，因为系统调度的原因，父进程的信号处理函数晚于子进程的信号处理函数是可能的。

这个现象我们在物理机上复现不了，关核也不行，但是在4核虚拟机上可以复现，这个跟虚拟机的实现原理应该有一定的关系，我们后续准备针对这块再深入研究一下。

现在，更好地理解内核的行为

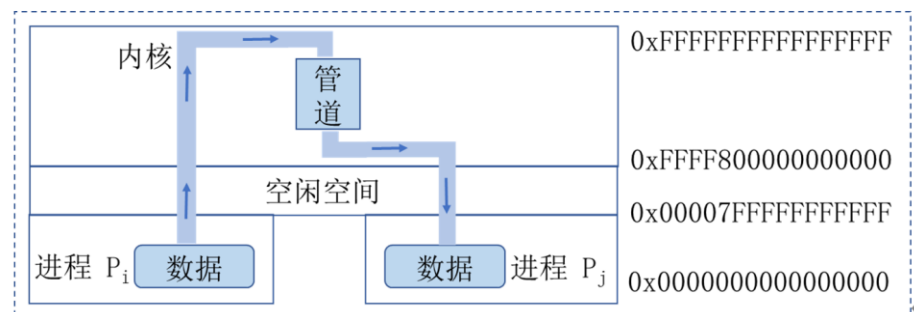
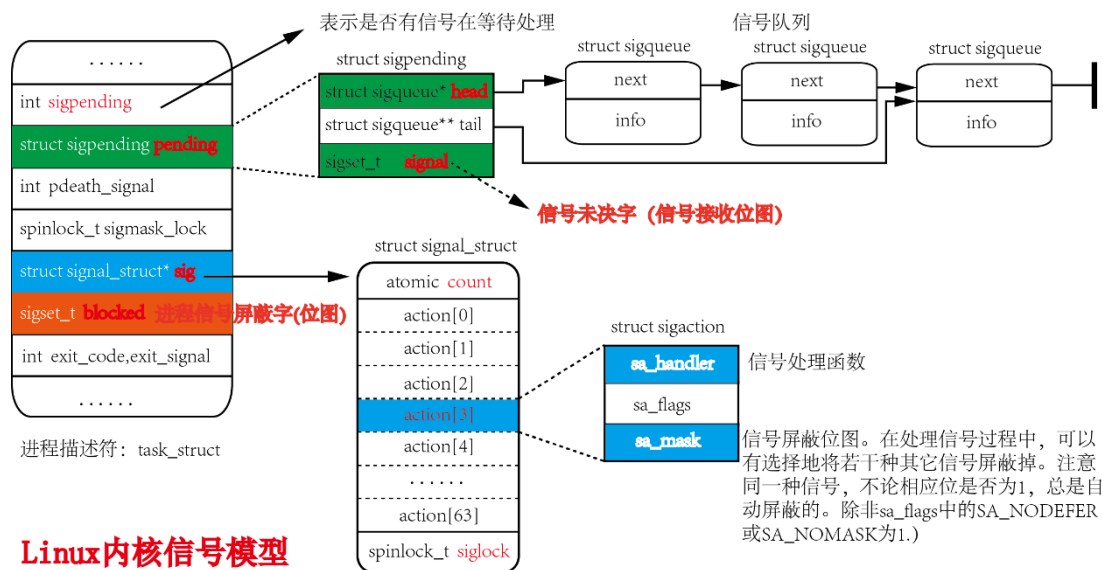
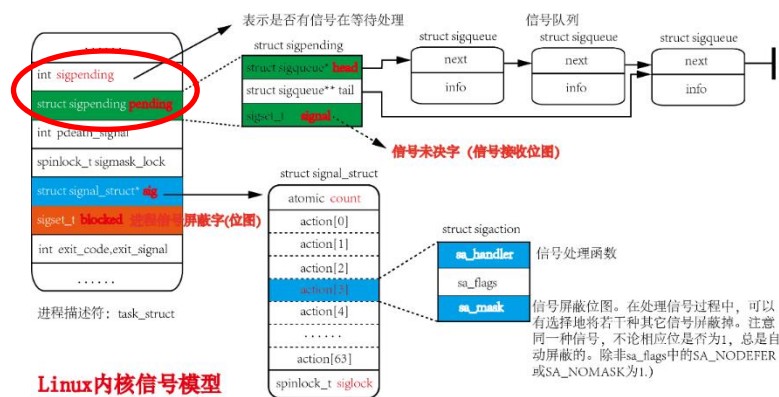


图 3-1 基于管道的进程间通信示意图



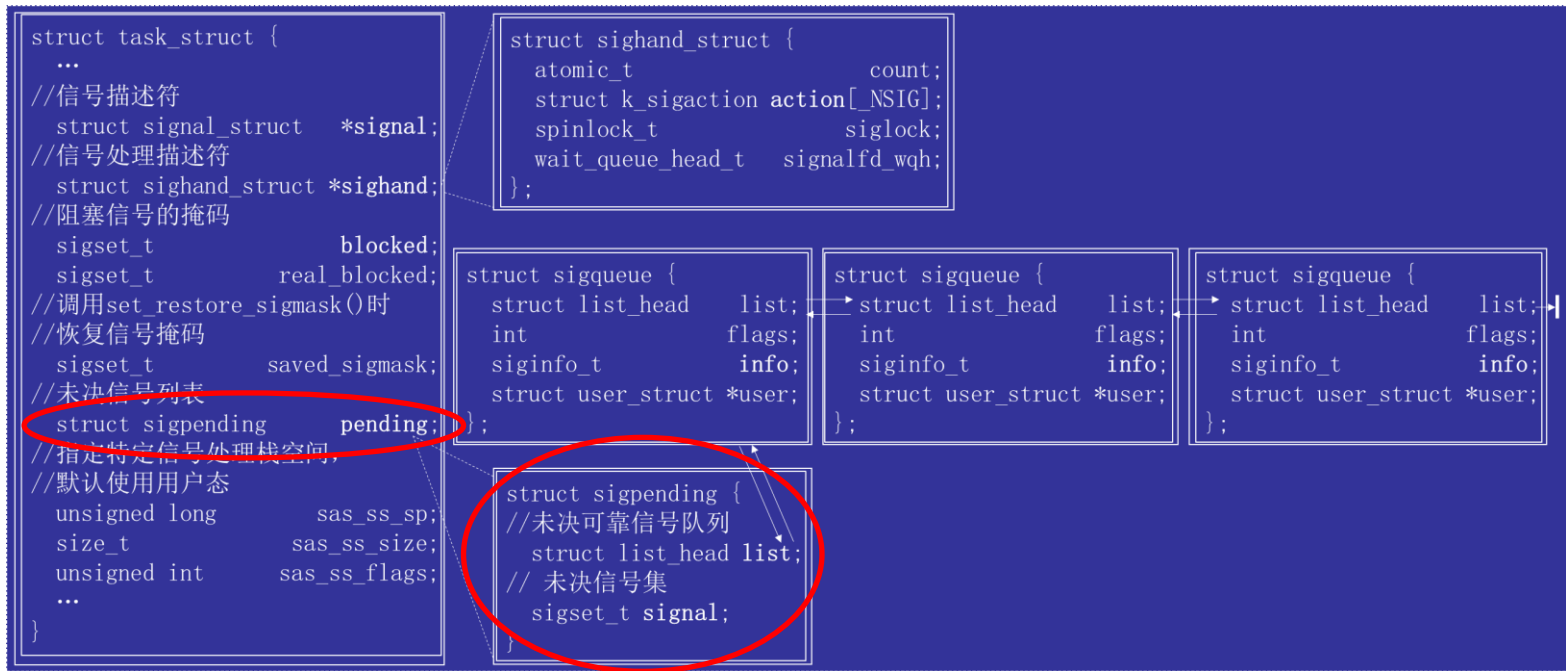
<https://www.cnblogs.com/5iedu/p/6401981.html>



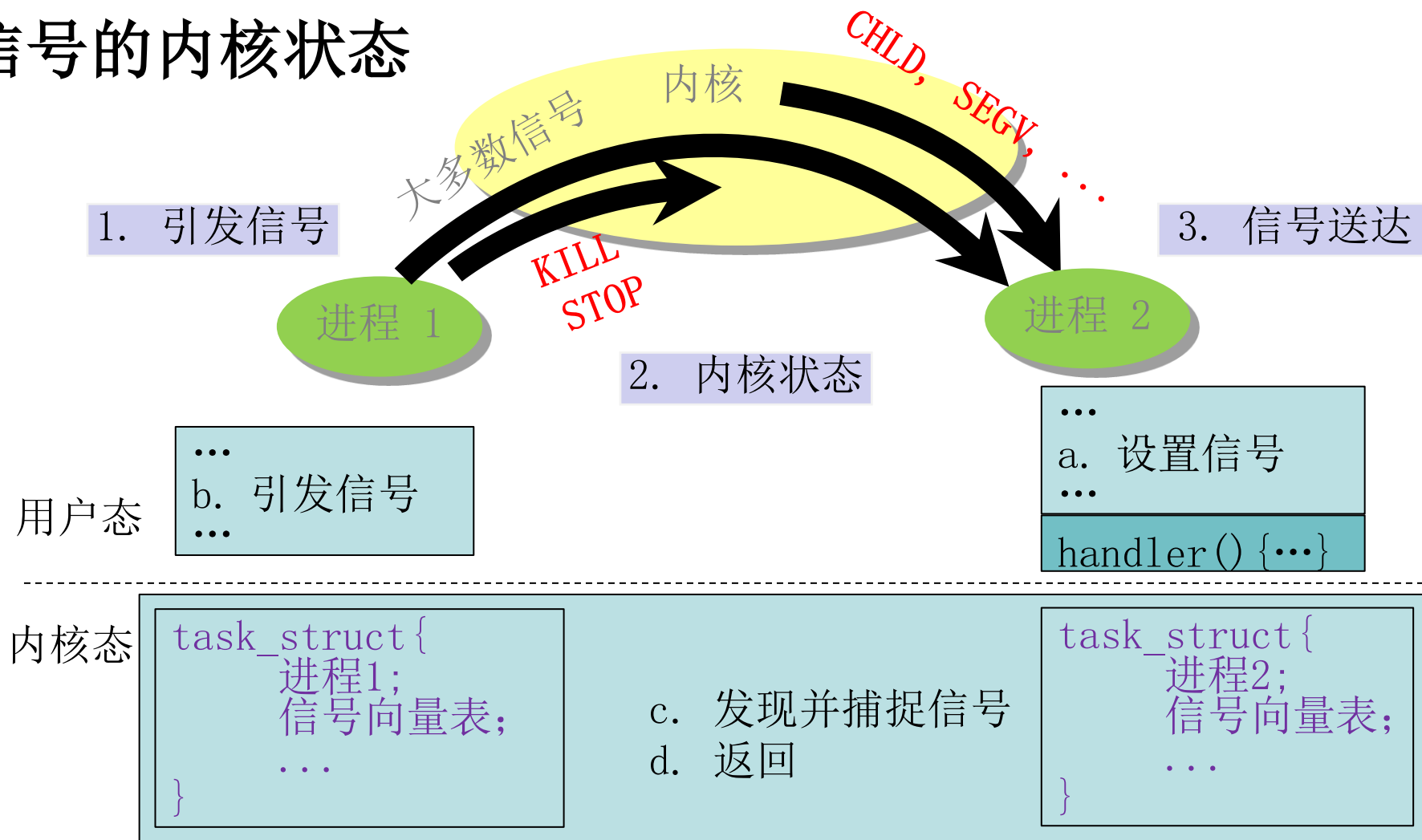
Linux内核信号模型

openEuler信号的内核模型是这样的 ...

2022.02.12



信号的内核状态

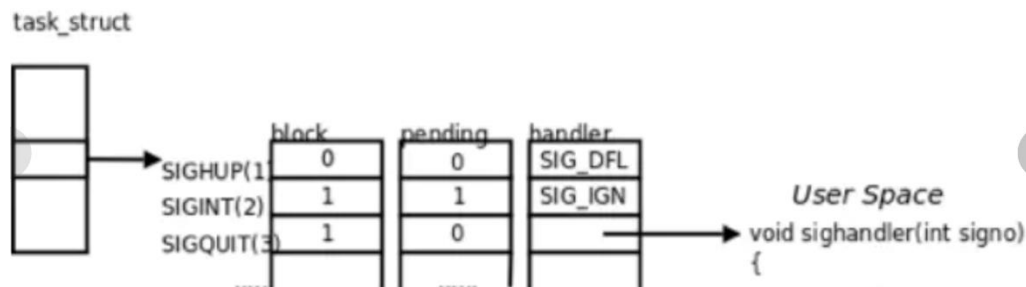


- 等待发送的信号集：已产生，还没有到达目的进程
- 被阻塞的信号集：继续等待；一旦解锁，送达
- 每个信号一个动作：信号到达后做什么

信号到达内核

■ 信号到达

- 为该信号设置等待位
- 每类信号仅一位



<https://www.jianshu.com/p/cfd8e9824f54>

■ 准备发送

- 选择一个等待、非阻塞信号执行其关联动作

- ◆ 忽略Ignore
- ◆ 终止进程

- ◆ 执行该进程设置的特定信号处理函数

■ 阻塞信号

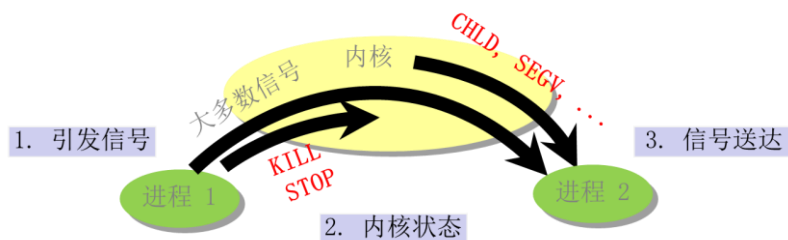
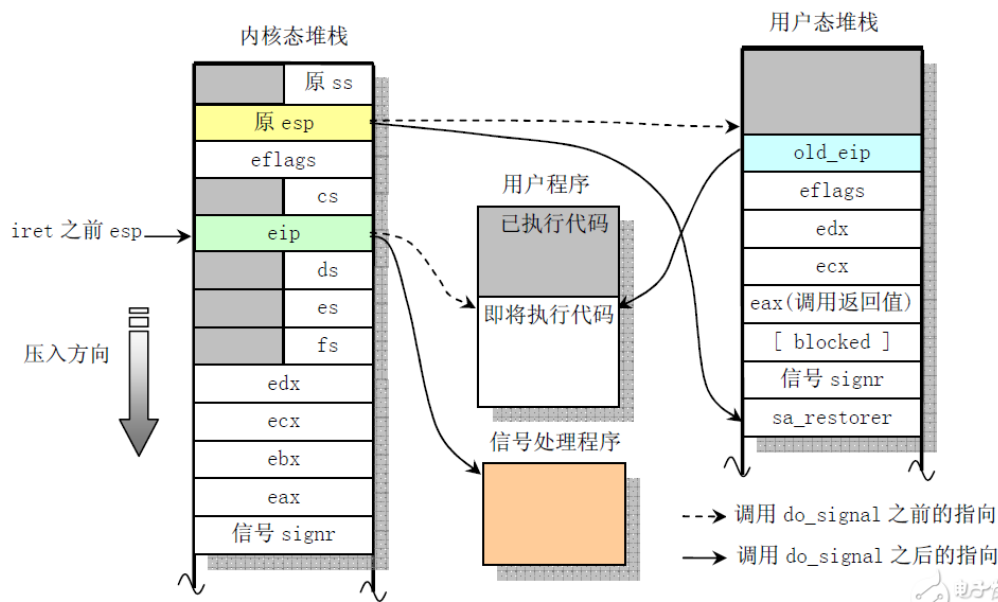
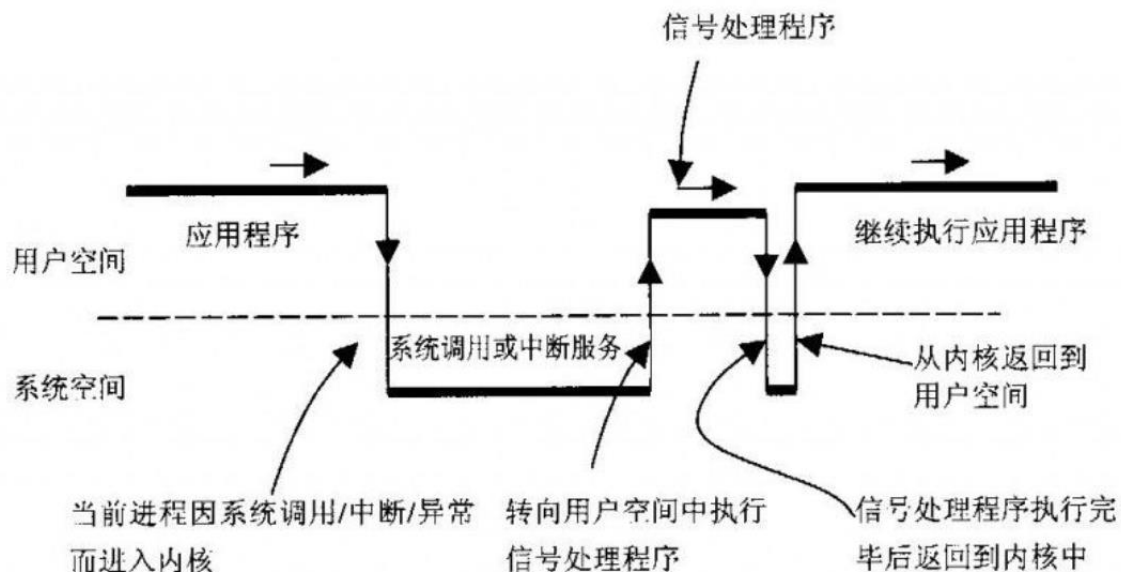
信号“送达”

■ 内核处理

- 不转发到目标进程
- 结束进程
 - `SIGSTOP`
 - `SIGKILL`

■ 目标进程处理

如果进程捕捉信号



图片来源: <https://blog.csdn.net/wangquan1992/article/details/108511628>

alarm() 系统调用

■ 最简单的定时器接口

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int n)
```

- n秒后, 将一个SIGALARM信号传递给调用进程
- 默认处理是终止进程
- k秒后再次调用, 新的闹钟取代原来的
 - ◆ 返回其剩下的秒数 $\text{MAX}((n-k), 0)$
 - ◆ 如果新调用参数为0, 前面设置的闹钟被取消,

pause()系统调用

- 等待信号(任何信号)

```
#include <unistd.h>
```

```
int pause(void)
```

- 调用进程进入可中断的休眠状态
- 内核调用schedule()调度新的进程执行

alarm() & pause() 应用例子

```
#include <sys/types.h>
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
```

```
void wakeup()
```

```
{
    printf("%ld: Alarm received from kernel\n",getpid());
}
```

```
int main()
```

```
{
    printf("%ld: about to sleep for 4 second\n",getpid());
```

```
    //将wakeup注册为我们用于处理SIGALRM的信号处理程序
```

```
    //SIGALRM:定时器到期
```

```
    if (signal(SIGALRM,wakeup)==SIG_ERR){
        fprintf(stderr,"Cannot handle SIGINT!\n");
        exit(EXIT_FAILURE);
    }
```

```
    //alarm()函数将SIGALRM信号送至调用它们的进程
```

```
    alarm(4);
```

```
    //让一个进程进入可中断的休眠状态，收到信号?处理信号：终止进程
```

```
    pause();
```

```
    printf("%ld: Morning so soon?\n",getpid());
```

```
[yuhong@FedoraDVD13 signal]$ ./alarm
27000: about to sleep for 4 second
27000: Alarm received from kernel
27000: Morning so soon?
[yuhong@FedoraDVD13 signal]$
```

僵尸进程的避免

1. 父进程调用`wait()`和`waitpid()`等函数等待子进程结束
2. 父进程调用`signal(SIGCHLD, handler)`函数为SIGCHLD注册处理函数`handler`
3. 父进程调用`signal(SIGCHLD, SIG_IGN)`通知内核忽略该信号
4. Stevens的两次fork，父进程和子进程退出，孙子进程被托孤


```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void sigchld_handler(int signum)
{
    int status;
    int pid = waitpid(-1, &status, WNOHANG);
    if ((pid > 0) && WIFEXITED(status)){
        printf("Child %d exited with code %d\n",pid,WEXITSTATUS(status));
    }
}
int main(int argc, char *argv[])
{
    pid_t cid;
    signal(SIGCHLD,sigchld_handler);
    cid = fork();
    if (cid > 0) while(1);
    else if (cid == 0) exit(0);
    return 0;
}

```

```

[proxyuser@taishan01 signals]$ ./sigchld_1 &
[1] 40593
[proxyuser@taishan01 signals]$ Child 40594 exited with code 0
kill -9 40593
[1]+  Killed                  ./sigchld_1
[proxyuser@taishan01 signals]$

```

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[])
{
    pid_t cid;
    signal(SIGCHLD, SIG_IGN);
    cid = fork();
    if (cid > 0) while(1);
    else if (cid == 0) exit(0);
}
```

signal方式的缺点

■ 缺信号来源

- 硬件错误？那个进程发的？

■ 处理函数运行过程中无法阻塞其他信息

sigaction() 系统调用

```
#include <signal.h>

int sigaction(int signum,
               const struct sigaction *act,
               struct sigaction *oldact);
```

参数说明

signum: 需要处理的特定信号

act: 非NULL时, 设置新的信号处理方式

oldact: 非NULL时, 保存原来的信号处理方式

(act == NULL) && (oldact != NULL): 查看进程原来的信号处理方式

sigaction 结构体定义

```
#include <signal.h>
```

```
struct sigaction{
```

```
    void (*sa_handler)(int);
```

```
    void (*sa_sigaction)(int, siginfo_t*, void*);
```

```
    sigset_t sa_mask; //信号处理函数运行期间被阻塞的信号集
```

```
    int sa_flags; //信号行为标志位
```

```
    void (*sa_restorer)(void); //已过时，POSIX不使用该成员
```

```
}
```

1. sa_flags==SA_SIGINFO时
2. 两者只能使用一个

- 有些体系结构采用联合体实现
 - sa_handler
 - sa_sigaction
- sa_handler同signal()函数的第2个参数，可以是
 - SIG_DFL
 - SIG_IGN
 - 用户定义的信号处理函数

sigaction 结构体定义

```
void (*sa_sigaction)(int, siginfo_t*, void*);
```

```
siginfo_t {  
    int si_signo; /* 信号描述符 */  
    int si_errno; /* 信号是否产生错误, 0: 无, 非0: 有, 即errno */  
    int si_code; /* 信号编码 */  
    pid_t si_pid; /* 发送进程ID */  
    uid_t si_uid; /* 发送进程的真实用户ID */  
    int si_status; /* Exit值或信号 */  
    ...  
}
```

signal↵	si_code↵	值↵	说明↵
SIGSEGV↵	SEGV_MAPERR↵	1↵	地址未映射到一个对象↵
	SEGV_ACCERR↵	2↵	对映射对象 <u>缺访问权限</u> ↵

参数3: 指向ucontext_t类型的指针

- 类型转换为void *
- 通常不用

sa_flags 可选值及其说明

■ 修改信号行为的标志位，可以用OR设置以下一位或多位的值

sa_flag	值	处理函数行为
0	0	等同于不设置系统默认行为
SA_NOCLDSTOP	0x00000001	当参数 <code>signum</code> 为 <code>SIGCHLD</code> 时，不接收子进程停止/续行等信号，只有当进程注册了 <code>SIGCHLD</code> 信号处理函数生效。
SA_NOCLDWAIT	0x00000002	当参数 <code>signum</code> 为 <code>SIGCHLD</code> 时，子进程终止时不进入僵死状态，只有当进程注册了 <code>SIGCHLD</code> 信号处理函数生效。
SA_RESETHAND	0x80000000	处理函数被调用时重置为缺省值 <code>SIG_DFL</code> ，注册了信号处理函数才生效。
SA_NODEFER	0x40000000	在信号处理函数运行时，内核不会阻塞该信号，注册了信号处理函数才生效。
SA_RESTART	0x10000000	由信号 <code>signum</code> 中断的系统调用会自动重启。
SA_SIGINFO	0x00000004	提供附加信息，一个指向 <code>siginfo_t</code> 数据结构的指针，但信号处理函数是用 <code>sa_sigaction</code> 设置生效。

sigaction() 应用例子（一）

sa_flags = 0

```
[szu@taishan02-vm-10 signal]$ gcc -Wall sa_0.c
[szu@taishan02-vm-10 signal]$ ./sa_0
Child kill SIGINT
OUCH!-I got signal 2
I will sleep for 3 seconds
I wake up.
Parent sleep another 10 minutes
Child kill SIGINT
OUCH!-I got signal 2
I will sleep for 3 seconds
Child kill SIGUSR1
Child kill SIGUSR2
User defined signal 2
[szu@taishan02-vm-10 signal]$
```

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6 void sigint_handler(int sig)
7 {
8     printf("OUCH!-I got signal %d\n",sig);
9     printf("I will sleep for 3 seconds\n");
10    sleep(3);
11    printf("I wake up.\n");
12 }
13 int main()
14 {
15     struct sigaction act;
16     act.sa_handler = sigint_handler;
17     sigemptyset(&act.sa_mask);
18     sigaddset(&act.sa_mask,SIGUSR1);
19     act.sa_flags = 0;
20     sigaction(SIGINT,&act,0);
21     pid_t cid = fork();
22     if (cid > 0){
23         wait(NULL);
24         printf("Parent sleep another 10 minutes\n");
25         sleep(10);
26         printf("Parent exits...\n");
27     } else if (cid == 0) {
28         printf("Child kill SIGINT\n");
29         kill(getppid(),SIGINT);
30         sleep(6);
31         printf("Child kill SIGINT\n");
32         kill(getppid(),SIGINT);
33         sleep(1);
34         printf("Child kill SIGUSR1\n");
35         kill(getppid(),SIGUSR1);
36         sleep(1);
37         printf("Child kill SIGUSR2\n");
38         kill(getppid(),SIGUSR2);
39     }
40 }
```

sigaction() 应用例子（二）

sa_flags = SA_RESETHAND

```
[szu@taishan02-vm-10 signal]$ gcc -Wall -o sa_resethand sa
[szu@taishan02-vm-10 signal]$ ./sa_resethand
Child kill SIGINT
OUCH!-I got signal 2
I will sleep for 3 seconds
I wake up.
Parent sleep another 10 minutes
Child kill SIGINT

[szu@taishan02-vm-10 signal]$ Child kill SIGUSR1
Child kill SIGUSR2
```

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6 void sigint_handler(int sig)
7 {
8     printf("OUCH!-I got signal %d\n",sig);
9     printf("I will sleep for 3 seconds\n");
10    sleep(3);
11    printf("I wake up.\n");
12 }
13 int main()
14 {
15     struct sigaction act;
16     act.sa_handler = sigint_handler;
17     sigemptyset(&act.sa_mask);
18     sigaddset(&act.sa_mask,SIGUSR1);
19     act.sa_flags = SA_RESETHAND;
20     sigaction(SIGINT,&act,0);
21     pid_t cid = fork();
22     if (cid > 0){
23         wait(NULL);
24         printf("Parent sleep another 10 minutes\n");
25         sleep(10);
26         printf("Parent exits...\n");
27     } else if (cid == 0) {
28         printf("Child kill SIGINT\n");
29         kill(getppid(),SIGINT);
30         sleep(6);
31         printf("Child kill SIGINT\n");
32         kill(getppid(),SIGINT);
33         sleep(1);
34         printf("Child kill SIGUSR1\n");
35         kill(getppid(),SIGUSR1);
36         sleep(1);
37         printf("Child kill SIGUSR2\n");
38         kill(getppid(),SIGUSR2);
39     }
40 }
```


sigaction() 应用例子（三）

sa_flags = SA_RESTART

```
[szu@taishan02-vm-10 signal]$ gcc -Wall -o sa
[szu@taishan02-vm-10 signal]$ ./sa_restart
Child kill SIGINT
OUCH!-I got signal 2
I will sleep for 3 seconds
I wake up.
Child kill SIGINT
OUCH!-I got signal 2
I will sleep for 3 seconds
Child kill SIGUSR1
Child kill SIGUSR2
User defined signal 2
[szu@taishan02-vm-10 signal]$
```

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6 void sigint_handler(int sig)
7 {
8     printf("OUCH!-I got signal %d\n",sig);
9     printf("I will sleep for 3 seconds\n");
10    sleep(3);
11    printf("I wake up.\n");
12 }
13 int main()
14 {
15     struct sigaction act;
16     act.sa_handler = sigint_handler;
17     sigemptyset(&act.sa_mask);
18     sigaddset(&act.sa_mask,SIGUSR1);
19     act.sa_flags = SA_RESTART;
20     sigaction(SIGINT,&act,0);
21     pid_t cid = fork();
22     if (cid > 0){
23         wait(NULL);
24         printf("Parent sleep another 10 minutes\n");
25         sleep(10);
26         printf("Parent exits...\n");
27     } else if (cid == 0) {
28         printf("Child kill SIGINT\n");
29         kill(getppid(),SIGINT);
30         sleep(6);
31         printf("Child kill SIGINT\n");
32         kill(getppid(),SIGINT);
33         sleep(1);
34         printf("Child kill SIGUSR1\n");
35         kill(getppid(),SIGUSR1);
36         sleep(1);
37         printf("Child kill SIGUSR2\n");
38         kill(getppid(),SIGUSR2);
39     }
40 }
```

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6 void sigint_handler(int sig, siginfo_t *info, void *context)
7 {
8     printf("Pid %d got sig %d from pid %d with uid %d\n", getpid(), sig, info->si_pid, info->si_uid);
9     printf("Pid %d will sleep for 3 seconds\n", getpid());
10    sleep(3);
11    printf("Pid %d wake up.\n", getpid());
12 }
13 int main()
14 {
15     struct sigaction act;
16     act.sa_sigaction = sigint_handler;
17     sigemptyset(&act.sa_mask);
18     act.sa_flags = SA_SIGINFO;
19     sigaction(SIGINT, &act, 0);
20     pid_t cid = fork();
21     if (cid > 0) {
22         wait(NULL);
23     } else if (cid == 0) {
24         printf("Child %d kill SIGINT\n", getpid());
25         kill(getppid(), SIGINT);
26     }
27 }
```

sigaction()
应用例子（四）

sa_flags = SA_SIGINFO

```
[szu@taishan02-vm-10 signal]$ gcc -Wall -o sa_siginfo sa_siginfo.c
[szu@taishan02-vm-10 signal]$ ./sa_siginfo
Child 422701 kill SIGINT
Pid 422700 got sig 2 from pid 422701 with uid 1000
Pid 422700 will sleep for 3 seconds
Pid 422700 wake up.
[szu@taishan02-vm-10 signal]$
```

应用练习题

■ 请编程序实现标准C库函数**sleep**（）

- 初步提示，相关系统调用

- ◆ alarm()

- ◆ pause()

- ◆ sigaction()/signal()

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
    /* nothing to do */
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1){
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}
```

思考

1. 信号处理函数sig_alm()什么都没干，能不注册吗？
2. mysleep()函数返回前恢复SIGALRM原来的sigaction, 为什么？
3. mysleep函数的返回值表示什么？什么情况下返回非0值？

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
```

```
void sig_alm(int signo)
{
    /* nothing to do */
}
```

```
unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1) {
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}
```

Q:信号处理函数sig_alm()
什么都没干，能不注册
吗？

```

#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    /* newact.sa_handler = sig_alm; */
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1){
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}

```

Q:信号处理函数sig_alm()
什么都没干，能不注册吗？

A:进程收到信号后会查找信号对应的处理函数并运行。如果不注册，则该函数指针为NULL，运行时出现Segmentation fault。

```

[yuhong@FedoraDVD13 homework2]$ gcc -o mysleep1 mysleep1.c
[yuhong@FedoraDVD13 homework2]$ ./mysleep1
段错误 (core dumped)

```

```

#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
    /* nothing to do */
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1) {
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}

```

Q:信号处理函数sig_alm()
什么都没干，能不注册吗？

A:进程收到信号后会查找注册信号对应的处理函数并运行。如果不注册，则该函数指针为NULL，运行时出现Segmentation fault。

```

[yuhong@FedoraDVD13 homework2]$ gcc -o mysleep1 mysleep1.c
[yuhong@FedoraDVD13 homework2]$ ./mysleep1
段错误 (core dumped)

```

Q: 信号处理函数 `sig_alm()`
什么都没干，能不注册
吗？

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    /* newact.sa_handler = sig_alm;
       sigemptyset(&newact.sa_mask);
       newact.sa_flags = 0;
       sigaction(SIGALRM,&newact,&oldact); */

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    /*sigaction(SIGALRM,&oldact,NULL); */

    return unslept;
}

int main(void)
{
    while(1){
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}
```

```
[yuhong@FedoraDVD13 homework2]$ gcc -o mysleep mysleep.c
[yuhong@FedoraDVD13 homework2]$ ./mysleep
闹钟
```



```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
```

```
void sig_alm(int signo)
{
    /* nothing to do */
}
```

```
unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1) {
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}
```

1. 信号处理函数 sig_alm() 什么都没干，能不注册吗？

```

#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
    /* nothing to do */
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1) {
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}

```

**Q:mysleep()函数返回前恢复
SIGALRM原来的sigaction,
为什么?**

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void sig_alm(int signo)
{
    /* nothing to do */
}

unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact, oldact;
    unsigned int unslept;

    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM, &newact, &oldact);

    alarm(nsecs);
    pause();

    unslept = alarm(0);
    sigaction(SIGALRM, &oldact, NULL);

    return unslept;
}

int main(void)
{
    while(1) {
        mysleep(2);
        printf("Two seconds passed\n");
    }
    return 0;
}
```

Q:mysleep函数的返回值表示什么？什么情况下返回非0值？

```
/*sig.c*/
```

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
```

```
void sig_alm(int signo){
    /*什么也不干*/
}
```

```
unsigned int mysleep(unsigned int nsecs){
    struct sigaction newact,oldact;
    unsigned int unslept;

    newact.sa_handler=sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags=0;
    sigaction(SIGALRM,&newact,&oldact);

    alarm(nsecs);
    //pause();|

    unslept=alarm(2);
    pause();
    sigaction(SIGALRM,&oldact,NULL);

    return unslept;
}
int main(){
    while(1){
        int i=mysleep(2);
        printf("Two seconds passed :%d\n",i);
    }
    return 0;
}
```



```
[chang_gui@localhost sigal]$ gcc sig.c
[chang_gui@localhost sigal]$ ./a.out
^C
[chang_gui@localhost sigal]$ gcc sig.c
[chang_gui@localhost sigal]$ ./a.out
Two seconds passed :2
Two seconds passed :2
Two seconds passed :2
^C
```

```
[yuhong@FedoraDVD13 signal]$ cat mysleepInterrupt.c
```

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
```

```
void sig_alm(int signo){ printf("get a signal\n"); }
void sig_others(int signo){ }
unsigned int mysleep(unsigned int nsecs)
{
    struct sigaction newact,oldact;
    unsigned int unslept;
    newact.sa_handler = sig_alm;
    sigemptyset(&newact.sa_mask);
    newact.sa_flags = 0;
    sigaction(SIGALRM,&newact,&oldact);
    pid_t childpid;
    signal(SIGPIPE,sig_others);
    signal(SIGINT,sig_others);
    childpid = fork();
    if (childpid < 0) {printf("Fork error.\n");exit(-1);}
    if (childpid>0){
        sleep(1);
        kill(childpid,SIGINT);
        wait(NULL);exit(0);
    }else{
        printf("The child process is yelling!\n");
        alarm(nsecs);
        pause();
    }
    unslept = alarm(0);
    printf("%d seconds left!\n",unslept);
    sigaction(SIGALRM,&oldact,NULL);
    return unslept;
}
int main(void)
{
    int unslept = mysleep(5);
    return 0;
}
```

```
[yuhong@FedoraDVD13 signal]$ ./mysleepInterrupt
The child process is yelling!
4 seconds left!
```

什么情况下返回非0值:

- 1) 后一个sleep取消了前一个, 则后一个sleep会返回非零值。
如sleep(3), sleep(0);
- 2) sleep被其他信号中断睡眠, 提前返回, 也会返回非零值。
- 3) 在调用出错的情况下返回-1。

```

#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>
#include "clientinfo.h"

#define FIFO_NAME "/tmp/server_fifo"
#define CLIENT2 "/tmp/client2_fifo"
#define BUFF_SZ 100

int main(int argc, char *argv[])
{
    int res;
    int fifo_fd, my_fifo;
    int fd;
    CLIENTINFO info;
    char buffer[BUFF_SZ];

    /*create my own FIFO */
    // sprintf(mypipename, "/tmp/client%d_fifo", getpid());
    res = mkfifo(CLIENT2, 0777);
    if(res != 0)
    {
        printf("FIFO %s was not created\n", buffer);
        exit(EXIT_FAILURE);
    }
}

```

```

/* open my own FIFO for reading */
my_fifo = open(CLIENT2, O_RDONLY | O_NONBLOCK);
if(my_fifo == -1)
{
    printf("Could not open %s for read only access\n",
        FIFO_NAME);
    exit(EXIT_FAILURE);
}

/* get result from server */
memset(buffer, '\0', BUFF_SZ);
while(1)
{
    res = read(my_fifo, buffer, BUFF_SZ);
    if(res > 0 )
    {
        printf("Received from server: %s\n", buffer);
        break;
    }
}

printf("Client %d is terminating\n", getpid());

/* delete fifo from system */
close(my_fifo);
(void)unlink(CLIENT2);

exit(0);

```

途中倒是好像发现了个小Bug，就是运行端2后再直接关闭，再运行端2和端1加服务器的时候，端2的管道好像就创建不了了（大概显示“FIFO XXX was not created”）。当时想出的方法是直接删除端2管道，运行下生成新的就好了。现在想想可能就是端2创建管道后，没运行服务器和端1传数据进来就关闭了，因为是堵塞的打开了读端，后面的代码就运行不了，倒数的一句void(unlink)CLIENT2就没执行。从而创建不了新的管道。

请为该同学支招